

ЗАНЯТИЕ 5 (2:00)

Теоретическое

ПРОДОЛЖИТЕЛЬНОСТЬ

Ориентировочная продолжительность занятия – 2 часа.

Механизм основных форм.....	156
Обработчики событий	157
Модули.....	158
Виды модулей	158
Контекст модуля формы.....	164
Форма как программный объект	174
Процедуры – обработчики событий в модуле формы	175
Как понять то, что написано в модуле формы	177
Как понять работу кода на встроенном языке	178
Анализ кода с помощью синтакс-помощника	180
Первый способ	180
Второй способ	191
Анализ кода с помощью отладчика	195
Объекты, объекты, объекты... ..	202
Сервер и клиенты.....	203
Компиляция общих модулей	207
Директивы компиляции.....	208
Исполнение кода на клиенте и на сервере	209

Чтобы не усложнять восприятие и без того объемного предыдущего занятия «Документы», все теоретические моменты были вынесены в это отдельное занятие. На нем будут подробно рассмотрены вопросы, которые мы опускали или бегло проходили ранее.

Эта теория очень важна для более глубокого понимания, почему мы делали так, а не иначе. Она необходима читателям, желающим в будущем самостоятельно разрабатывать конфигурации, а не просто шаг за шагом следовать данному пособию.

Однако если прямо сейчас вы не в состоянии усвоить этот раздел, можно его пропустить и вернуться к нему позднее. На выполнение сквозного примера разработки учебной конфигурации, рассмотренного в данной книге, это никак не повлияет. Итак...

Механизм основных форм

На предыдущем занятии мы создали форму документа Приходная накладная и назначили эту форму основной. Что это значит?

У всех прикладных объектов конфигурации существует некоторое количество основных форм. Они служат для отображения данных объекта в том или ином виде.

Если разработчик не назначит в качестве основных форм объекта свои собственные, система будет генерировать необходимые формы объекта самостоятельно, в те моменты, когда к ним происходит обращение.

Наличие такого механизма позволяет разработчику не тратить время на создание форм для тестирования своей разработки, а воспользоваться тем, что платформа создаст по умолчанию.

Создание этих форм происходит динамически, в процессе работы системы. Форма создается в тот момент, когда к ней происходит обращение. Причем не важно, интерактивное это обращение или программное.

Так, форма списка для справочника Клиенты будет создана как при интерактивном выборе в меню Функции для технического специалиста > Справочники > Клиенты, так и при программном вызове глобального метода ОткрытьФорму() (листинг 5.1).

Листинг 5.1. Программный вызов метода «ОткрытьФорму()»

```
ОткрытьФорму("Справочник.Клиенты.ФормаСписка");
```

Также примечательным фактом является то, что состав основных форм, определенных для объекта конфигурации, может не совпадать с перечнем тех форм, которые вообще можно создать для данного объекта, используя конструктор формы.

Например, для большинства регистров в конфигураторе можно задать основную форму списка. Однако если открыть конструктор форм для регистра, вы увидите, что кроме формы списка предлагается создать и форму набора записей регистра, которая отсутствует в перечне основных форм.

Дело в том, что состав основных форм определяется исходя из того, какое представление данных может понадобиться в процессе интерактивной работы пользователя. Для этих представлений разработчик может создать свои формы и указать их в качестве основных, а может использовать те формы, которые система создаст автоматически.

Конструктор форм, напротив, исходит из потребностей разработчика. Если разработчик посчитает нужным использовать для какого-либо регистра вместо обычной формы списка форму набора записей, он сможет это сделать, воспользовавшись конструктором и определив ее в качестве основной формы регистра. Но для логики работы системы это не будет иметь принципиального значения.

Обработчики событий

На предыдущем занятии мы создавали обработчики событий ПриИзменении у некоторых элементов формы. Что это такое?

При работе с событиями на платформе «1С:Предприятие» следует различать два типа событий: события, связанные с формой и ее элементами, и все остальные.

Разница заключается в том, что обработчики событий, связанных с формой и ее элементами, — назначаемые, а обработчики всех остальных событий — фиксированные.

Фиксированный обработчик события должен иметь имя, совпадающее с именем события. Только в этом случае он будет вызываться при возникновении соответствующего события.

Назначаемый обработчик может иметь произвольное имя. Если имя процедуры совпадает с именем события формы или ее элемента, этого совсем недостаточно для вызова процедуры обработки события с таким именем. Требуется явное назначение процедуры обработчиком этого события в палитре свойств, в соответствующем событии.

Таким образом, любая процедура, расположенная в модуле формы, может быть назначена обработчиком любого события (или сразу нескольких событий) формы или ее элемента, расположенного в форме. Имя процедуры в этом случае не имеет значения. Важно лишь то, что она назначена обрабатывать какое-либо событие.

Назначение обработчика может выполняться интерактивно, при работе с формой в конфигураторе, или программно, используя методы формы и ее элементов – `УстановитьДействие()`.

Модули

На предыдущем занятии мы рассматривали код обработчиков событий. Мы узнали, что эти процедуры располагаются в модуле формы – некоем хранилище текста программы на встроенном языке.

Теперь расскажем о модулях подробнее и внимательнее познакомимся с устройством модуля формы.

Виды модулей

В конфигурации существуют различные виды модулей. Они могут принадлежать некоторым объектам конфигурации (например, формам), а могут существовать сами по себе (принадлежать всей конфигурации в целом).

Текст программы, содержащийся в модулях, будет использоваться платформой в заранее известные моменты работы системы «1С:Предприятие» – *события*, о которых мы рассказывали ранее.

В «1С:Предприятии» существуют следующие виды модулей.

Модуль управляемого приложения. Модуль управляемого приложения выполняется при старте и окончании работы системы «1С:Предприятие» в режимах тонкого клиента и веб-клиента.

В нем возможно объявление переменных, а также объявление и описание процедур и функций, которые будут доступны в любом модуле конфигурации (кроме модуля внешнего соединения). Их доступность также обеспечивается для неглобальных общих модулей с установленным свойством Клиент (управляемое приложение). В контексте модуля управляемого приложения доступны экспортируемые процедуры и функции общих модулей.

Чтобы открыть модуль управляемого приложения, нужно выделить корень дерева объектов конфигурации (строку ПособиеДляНачинающих) и вызвать из контекстного меню команду Открыть модуль управляемого приложения (рис. 5.1).

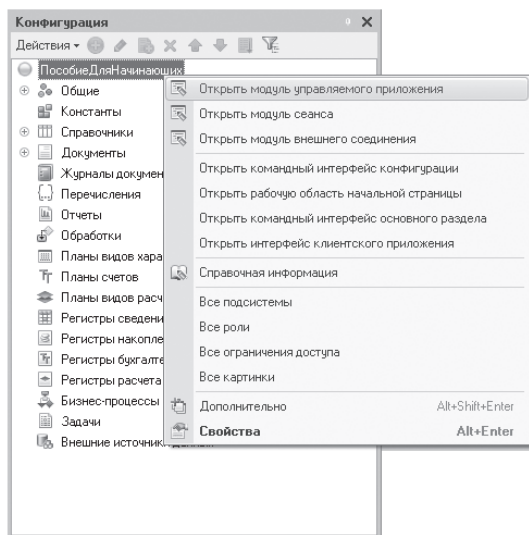


Рис. 5.1. Открытие модуля управляемого приложения

Общие модули. В общих модулях хранятся процедуры и функции, которые вызываются из других модулей системы. Сам по себе общий модуль не исполняется. Исполняются отдельные его процедуры/функции в момент их вызова из других модулей.

Чтобы открыть общий модуль, нужно раскрыть ветвь Общие в дереве объектов конфигурации, затем раскрыть ветвь Общие модули и дважды щелкнуть мышью на нужном модуле (рис. 5.2).

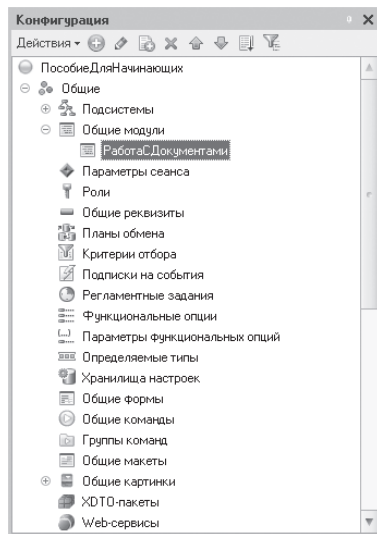


Рис. 5.2. Открытие общего модуля

Модули объектов. Модули объектов – это, например, модуль элемента справочника или модуль документа.

Эти модули вызываются тогда, когда программно создается этот объект средствами встроенного языка, например, методами `СоздатьЭлемент()` менеджеров справочников или `СоздатьДокумент()` менеджеров документов, либо когда пользователь создает новый элемент справочника или документ интерактивно.

При записи измененных данных объекта в базу данных вызываются различные обработчики событий, которые располагаются в модуле объекта.

Более подробно о последовательности событий при записи объектов конфигурации в базу данных рассказывается в разделе «Краткий справочник разработчика».

Чтобы открыть модуль объекта, нужно в окне редактирования объекта конфигурации перейти на закладку *Прочее* и нажать кнопку *Модуль объекта* (рис. 5.3). Или, выделив нужный объект в дереве объектов конфигурации, вызвать из контекстного меню команду *Открыть модуль объекта*.

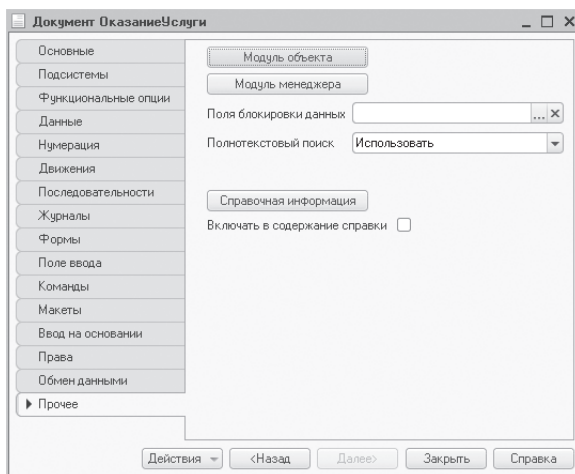


Рис. 5.3. Открытие модуля объекта

Модули форм. Каждая форма, определенная в конфигурации, имеет свой собственный модуль. Этот модуль выполняется при создании объекта `УправляемаяФорма` встроенного языка. Этот объект создается системой в режиме 1С:Предприятие в тот момент, когда мы программно (методом `ОткрытьФорму()`) или интерактивно открываем форму некоторого элемента.

Чтобы открыть модуль формы, нужно открыть подчиненный объект `Форма` нужного объекта конфигурации и в окне редактора форм перейти на закладку `Модуль` (рис. 5.4).

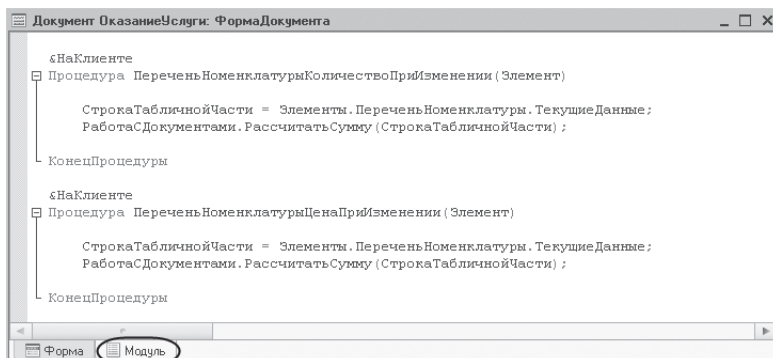


Рис. 5.4. Открытие модуля формы

Модуль сеанса. Модулем сеанса называется модуль, который автоматически выполняется при старте системы «1С:Предприятие» в момент загрузки конфигурации. Модуль сеанса предназначен для инициализации параметров сеанса и обработки действий, связанных с сеансом работы. Модуль сеанса не содержит экспортируемых процедур и функций и может использовать процедуры из общих модулей конфигурации.

Чтобы открыть модуль сеанса, нужно выделить корень дерева объектов конфигурации (строку *ПособиеДляНачинающих*) и вызвать из контекстного меню команду *Открыть модуль сеанса* (рис. 5.5).

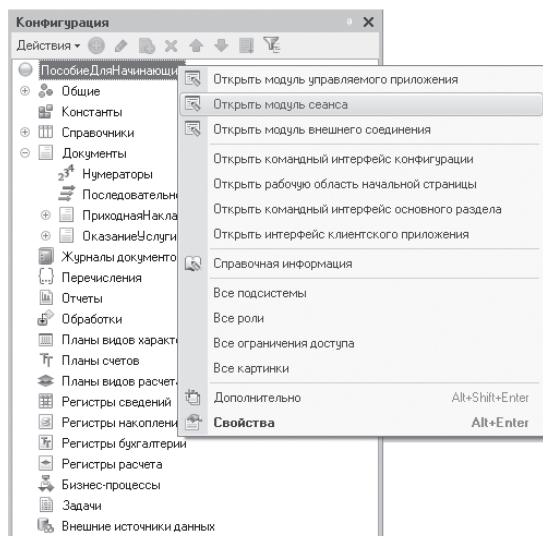


Рис. 5.5. Открытие модуля сеанса

Модуль внешнего соединения предназначен для размещения в нем текстов функций и процедур, которые могут вызываться в сессии внешнего соединения.

Чтобы открыть модуль внешнего соединения, нужно выделить корень дерева объектов конфигурации (строку *ПособиеДляНачинающих*) и вызвать из контекстного меню команду *Открыть модуль внешнего соединения* (см. рис. 5.5).

Модуль менеджера. Для каждого прикладного объекта существует менеджер, предназначенный для управления этим объектом как объектом конфигурации. С помощью менеджера можно создавать объекты, работать с формами и макетами. Модуль менеджера позволяет расширить функциональность менеджеров, предоставляемых системой, за счет написания процедур и функций на встроенном языке.

Фактически это позволяет описать собственные методы для объекта конфигурации (например, справочника), которые относятся не к конкретному экземпляру объекта базы данных, а к самому объекту конфигурации. Например, в модуле менеджера можно задать произвольное представление объекта в интерфейсе «1С:Предприятия».

Чтобы открыть модуль менеджера, нужно в окне редактирования объекта конфигурации перейти на закладку Прочее и нажать кнопку Модуль менеджера (рис. 5.6). Или, выделив нужный объект в дереве объектов конфигурации, вызвать из контекстного меню команду Открыть модуль менеджера.

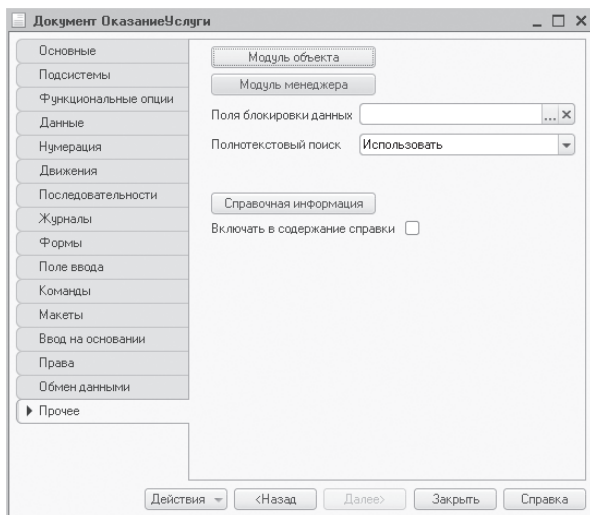


Рис. 5.6. Открытие модуля менеджера

Модуль команды. Как в самой конфигурации, так и у многих прикладных объектов, могут существовать подчиненные объекты

конфигурации – *Команды*. У каждой команды существует модуль команды, в котором можно написать предопределенную процедуру `ОбработкаКоманды()` для выполнения этой команды.

Чтобы открыть модуль команды, подчиненной некоторому объекту конфигурации, нужно в окне редактирования объекта конфигурации перейти на закладку *Команды* и дважды щелкнуть мышью на нужной команде (рис. 5.7). Или, выделив нужную команду в дереве объектов конфигурации, вызвать из контекстного меню команду *Открыть модуль команды*.

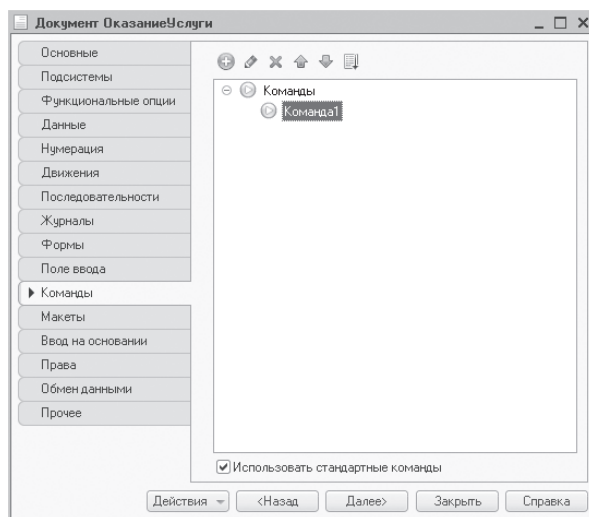


Рис. 5.7. Открытие модуля команды объекта конфигурации

Контекст модуля формы

Каждый модуль связан с остальной частью конфигурации. Эта связь называется *контекстом модуля*.

Контекст модуля определяет набор доступных во время выполнения модуля объектов, переменных, процедур и функций.

Поскольку дальше речь пойдет о том обработчике, который мы написали в модуле формы, рассмотрим подробнее, из чего складывается контекст модуля формы.

Контекст модуля формы образуется:

- локальным контекстом самого модуля формы;
- реквизитами формы, которой «принадлежит» модуль;
- свойствами и методами объекта `УправляемаяФорма` встроенного языка;
- свойствами и методами расширения формы, определяемого типом того объекта, данные которого содержатся в основном реквизите формы;
- глобальным контекстом, в том числе неглобальными общими модулями и экспортируемыми функциями и процедурами глобальных общих модулей;
- экспортируемыми переменными, процедурами и функциями модуля управляемого приложения.

Рассмотрим подробнее, что собой представляет каждая из перечисленных составляющих.

1. Локальный контекст модуля формы.

Локальный контекст модуля формы – это переменные, процедуры и функции, объявленные в этом модуле.

Например, внутри модуля формы можно непосредственно обращаться по имени к процедуре `ПолучитьСумму()`, объявленной в этом же модуле (листинг 5.2):

Листинг 5.2. Модуль формы

```
&НаКлиенте
Процедура Команда1()
    ПолучитьСумму();
КонецПроцедуры

&НаСервереБезКонтекста
Процедура ПолучитьСумму()
    ...
КонецПроцедуры
```

Или внутри модуля формы можно непосредственно обращаться по имени к переменной `СлужебнаяПеременная`, объявленной в этом модуле (листинг 5.3).

Листинг 5.3. Модуль формы

```

&НаКлиенте
Перем СлужебнаяПеременная;

&НаКлиенте
Процедура Команда1()
    СлужебнаяПеременная = 3;
КонецПроцедуры

```

2. Реквизиты формы, которой «принадлежит» модуль.

Например, если у формы существует реквизит СлужебныйРеквизит (рис. 5.8), то к нему можно непосредственно обращаться по имени (листинг 5.4):

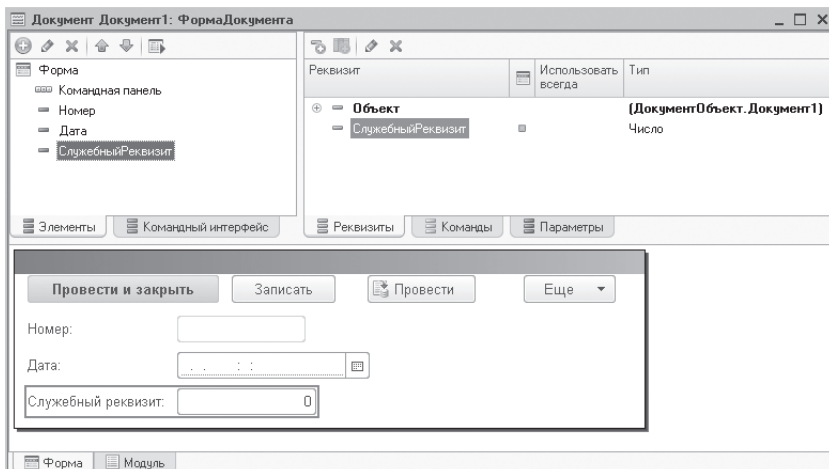


Рис. 5.8. Реквизит формы – «Служебный реквизит»

Листинг 5.4. Модуль формы

```

&НаКлиенте
Процедура Команда1()
    СлужебныйРеквизит = 3;
КонецПроцедуры

```

3. Свойства и методы объекта «УправляемаяФорма».

Свойства и методы объекта встроенного языка УправляемаяФорма описаны в синтакс-помощнике: Интерфейс (управляемый) > УправляемаяФорма (рис. 5.9).

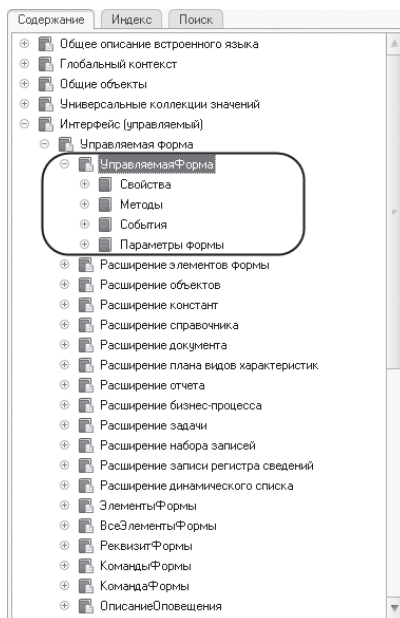


Рис. 5.9. Свойства управляемой формы в синтаксис-помощнике

К ним можно обращаться непосредственно по имени. Например, можно задать заголовок формы (листинг 5.5).

Листинг 5.5. Модуль формы

```
&НаКлиенте  
Процедура Команда1()  
    Заголовок = "Новый заголовок формы";  
КонiecПроцедуры
```

Или можно закрыть форму (листинг 5.6).

Листинг 5.6. Модуль формы

```
&НаКлиенте  
Процедура Команда1()  
    Закреть();  
КонiecПроцедуры
```

- 4. Свойства и методы расширения формы, определяемого типом того объекта, данные которого содержатся в основном реквизите формы.**

Один из реквизитов формы может быть основным, в списке реквизитов он выделяется жирным. Как правило, основной реквизит формы содержит данные того объекта, который отображается в форме. Например, если это форма справочника, то основной реквизит будет содержать данные объекта СправочникОбъект.<имя> (рис. 5.10).

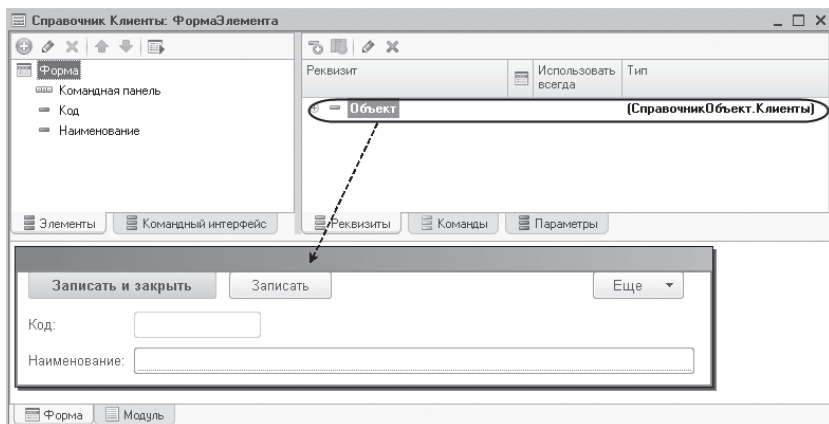


Рис. 5.10. Основной реквизит формы

А если это форма документа, то основной реквизит формы будет содержать данные объекта ДокументОбъект.<имя> (рис. 5.11).

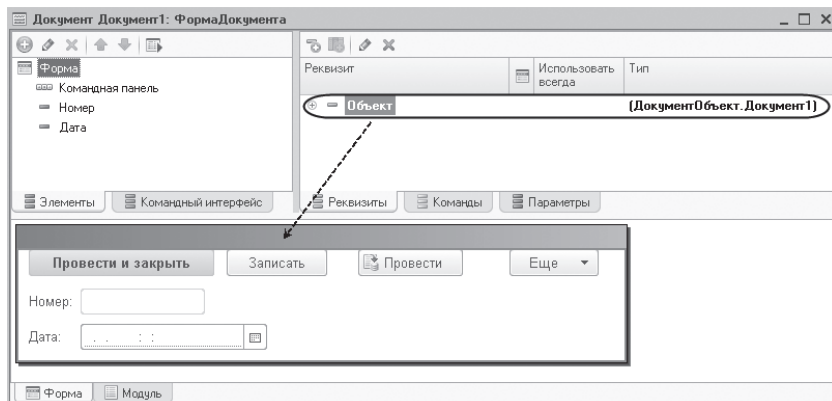


Рис. 5.11. Основной реквизит формы

Тут следует пояснить, почему в списке реквизитов тип основного реквизита указан в скобках: (ДокументОбъект.Документ1). Дело в том, что это «ненастоящий» тип реквизита формы. Настоящий тип в данном случае будет ДанныеФормыСтруктура (рис. 5.12).

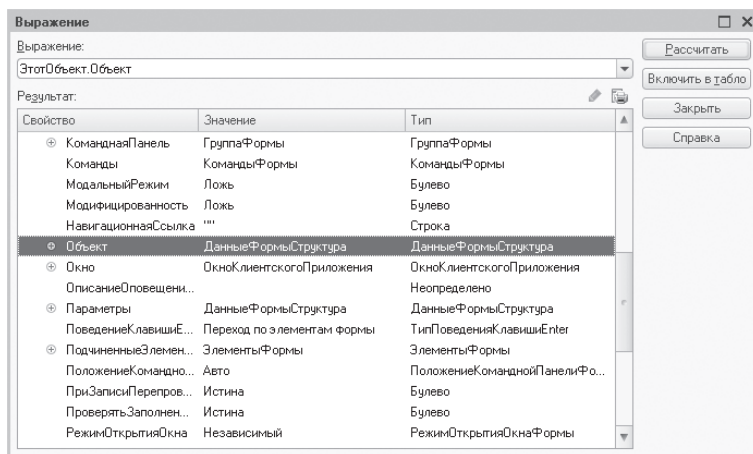


Рис. 5.12. Тип объекта основного реквизита формы

Но **ДанныеФормыСтруктура** – это универсальный тип, который может содержать данные различных прикладных объектов. Поэтому, чтобы в редакторе формы можно было «ориентироваться», данные какого же именно прикладного объекта отображает эта форма, в колонке Тип редактора показывается не тип реквизита формы (**ДанныеФормыСтруктура**), а тип того объекта, данные которого содержатся в этом реквизите. И этот «ненастоящий» тип показывается в скобках.

От типа объекта, данные которого содержатся в основном реквизите формы, зависят некоторые особенности в поведении формы.

Например, если основной реквизит формы будет содержать данные документа, то при закрытии такой формы система будет запрашивать подтверждение записи и проведения этого документа. Если же основной реквизит формы будет содержать данные справочника, то подобного запроса подтверждения при закрытии формы возникать не будет.

В зависимости от того, какой тип имеет объект, данные которого содержатся в основном реквизите, к контексту программного объекта **УправляемаяФорма** добавляется контекст соответствующего расширения.

Например, если основной реквизит – **СправочникОбъект.<имя>**, то в модуле формы становятся доступны свойства, методы объекта встроенного языка **Расширение управляемой формы для справочника** (синтакс-помощник – **Интерфейс (управляемый) > Управляемая форма > Расширение справочника**), рис. 5.13.

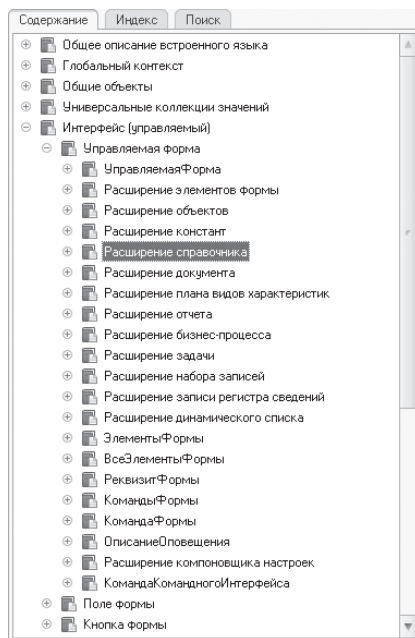


Рис. 5.13. Описание объектов в синтакс-помощнике

А если основной реквизит – **ДинамическийСписок** (рис. 5.14), то в модуле формы становятся доступны свойства, методы объекта встроенного языка **Расширение управляемой формы для динамического списка** (синтакс-помощник – **Интерфейс (управляемый) > Управляемая форма > Расширение динамического списка**), рис. 5.15.

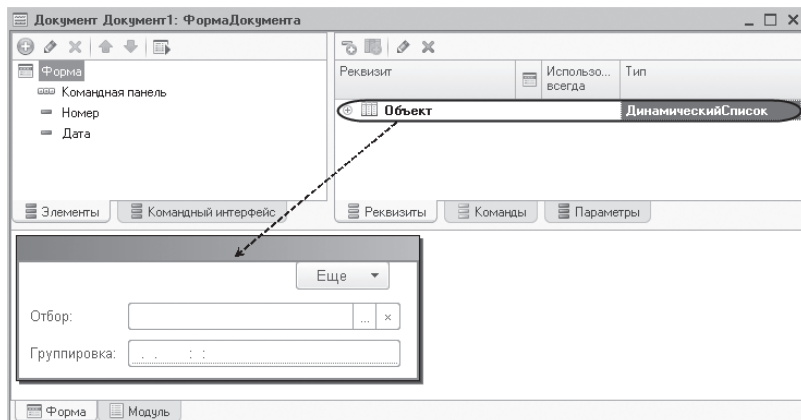


Рис. 5.14. Основной реквизит формы

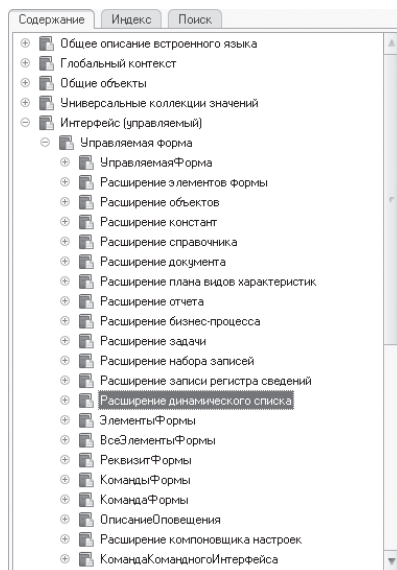


Рис. 5.15. Описание объектов в синтаксис-помощнике

Таким образом, в модуле формы, где основной реквизит содержит данные документа (рис. 5.16), можно обратиться к свойству расширения управляемой формы для документа АвтоВремя (листинг 5.7).

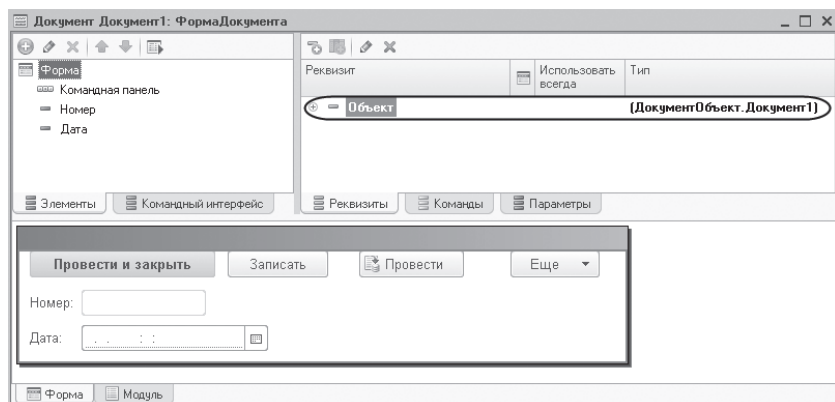


Рис. 5.16. Основной реквизит формы

Листинг 5.7. Модуль формы

```
&НаКлиенте
Процедура Команда1()
    АвтоВремя = РежимАвтоВремя.Первым;
КонецПроцедуры
```

Или записать документ с помощью метода расширения управляемой формы для документа **Записать()**, листинг 5.8:

Листинг 5.8. Модуль формы

```
&НаКлиенте
Процедура Команда1()
    Записать(РежимЗаписиДокумента.Проведение);
КонецПроцедуры
```

5. Глобальный контекст, в том числе неглобальные общие модули и экспортируемые функции и процедуры глобальных общих модулей.

В модуле формы можно получить системную дату, обратившись к встроенной функции **ТекущаяДата()**, листинг 5.9.

Листинг 5.9. Модуль формы

```
&НаКлиенте
Процедура Команда1()
    Сообщить(ТекущаяДата());
КонецПроцедуры
```

Или получить историю работы пользователя, обратившись к свойству глобального контекста `ИсторияРаботыПользователя` (листинг 5.10).

Листинг 5.10. Модуль формы

```
&НаКлиенте  
Процедура Команда1()  
    История = ИсторияРаботыПользователя.Получить();  
КонечПроцедуры
```

Если в глобальном общем модуле (например, `ОбменДанными`) определена экспортная процедура `ПолучитьПрефиксНомера()` (листинг 5.11), то в модуле формы можно обращаться к ней по имени (листинг 5.12).

Листинг 5.11. Глобальный общий модуль

```
Функция ПолучитьПрефиксНомера() Экспорт  
    Возврат Константы.ПрефиксНумерации.Получить();  
КонечФункции
```

Листинг 5.12. Модуль формы

```
&НаКлиенте  
Процедура Команда1(Префикс)  
    Префикс = ПолучитьПрефиксНомера();  
КонечПроцедуры
```

Если такой общий модуль – неглобальный (например, `РаботаСДокументами`), то при обращении к процедуре ее имя нужно указывать через точку от имени модуля (листинг 5.13).

Листинг 5.13. Модуль формы

```
&НаКлиенте  
Процедура КоличествоПриИзменении(Элемент)  
    РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);  
КонечПроцедуры
```

Второй способ предпочтительнее, так как неглобальные общие модули компилируются по мере обращения к ним, а не при запуске системы, как глобальные.

Естественно, при этом нужно обеспечивать согласованность того, как описана процедура в модуле формы (`&НаКлиенте`, `&НаСервере` и т.д.), и того, какие флажки проставлены у общего модуля (клиент (управляемое приложение), сервер и т.д.).

6. Экспортируемые переменные, процедуры и функции модуля управляемого приложения.

Если в модуле приложения определена экспортная процедура `ТестовоеСообщение()` (листинг 5.14), то в модуле формы можно обращаться к ней по имени (листинг 5.15).

Листинг 5.14. Модуль приложения

```
Процедура ТестовоеСообщение() Экспорт  
    Сообщить("Тестовое сообщение");  
КонецПроцедуры
```

Листинг 5.15. Модуль формы

```
&НаКлиенте  
Процедура Команда1()  
    ТестовоеСообщение();  
КонецПроцедуры
```

Форма как программный объект

Помимо того что форма внутри своего модуля предоставляет доступ к различным частям конфигурации, она также доступна из других частей конфигурации как программный объект.

При этом помимо стандартных свойств и методов объекта встроенного языка `УправляемаяФорма` у нее могут существовать и другие свойства и методы, определенные разработчиком.

Например, если в модуле формы `ФормаДокумента` документа `ПриходнаяНакладная` описана экспортируемая процедура `МатериалыКоличествоПриИзменении()` (листинг 5.16), то может быть использован следующий вызов этой процедуры (листинг 5.17).

Листинг 5.16. Модуль формы

```
&НаКлиенте  
Процедура МатериалыКоличествоПриИзменении(Элемент)  
    СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные;  
    РаботаСДокументами.РассчитатьСумму(СтрокаТабличнойЧасти);  
КонецПроцедуры
```

Листинг 5.17. Модуль формы

```
Форма = ПолучитьФорму("Документ.ПриходнаяНакладная.Форма.ФормаДокумента");  
Форма.МатериалыКоличествоПриИзменении();
```

Процедуры – обработчики событий в модуле формы

Помимо описания переменных и основной программы, модуль формы может содержать описание процедур – обработчиков событий, связанных с формой. Основными событиями, которые могут обрабатываться в модуле формы, являются события открытия и закрытия окна формы (листинг 5.18).

Листинг 5.18. Модуль формы

```
&НаСервере  
Процедура ПриСозданииНаСервере(Отказ, СтандартнаяОбработка)  
    // Вставить содержимое обработчика.  
КонецПроцедуры  
  
&НаКлиенте  
Процедура ПриОткрытии(Отказ)  
    // Вставить содержимое обработчика.  
КонецПроцедуры
```

Важным моментом здесь является то, что имена этих процедур не фиксированы, они могут иметь произвольные имена. Поэтому недостаточно, например, написать в модуле определения процедур с именами `ПриСозданииНаСервере` или `ПриОткрытии`. Кроме этого, их обязательно нужно связать в конфигураторе с соответствующими событиями формы (рис. 5.17).

Начинающие разработчики зачастую забывают об этом. Чтобы не попадать в такие ситуации, для автоматического создания таких процедур лучше использовать кнопки открытия  со значком лупы в панели свойств (рис. 5.18).

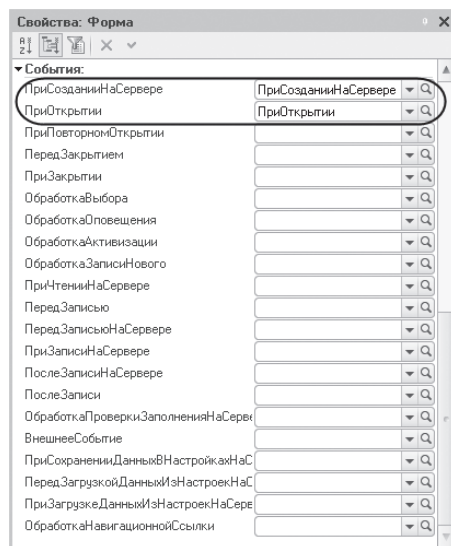


Рис. 5.17. События формы

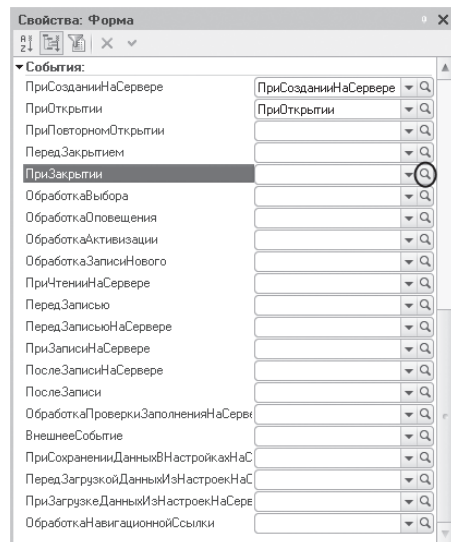


Рис. 5.18. События формы

Или можно выбрать их из выпадающего списка, доступного, когда вы находитесь в модуле формы (рис. 5.19).

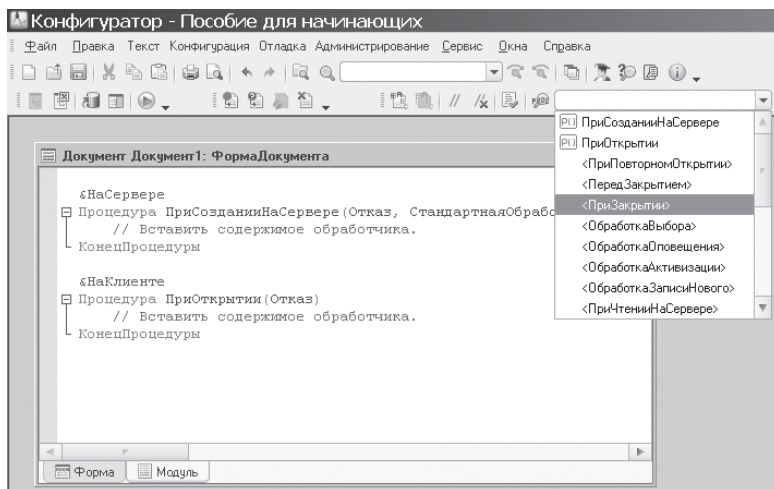


Рис. 5.19. Список событий формы

В этом случае система автоматически создаст определения этих процедур и свяжет их с соответствующими свойствами формы.

Как понять то, что написано в модуле формы

В заключение кратко опишем общий алгоритм, позволяющий разобраться с кодом, написанным в модуле формы. Рассматривать его будем на конкретном примере нашего обработчика события `МатериалыКоличествоПриИзменении`.

Допустим, в модуле формы нам встретилось выражение: `СтрокаТабличнойЧасти = ЭлементыФормы.Материалы.ТекущиеДанные`.

Как понять, что такое `СтрокаТабличнойЧасти`? Нужно вспомнить, из чего состоит контекст формы:

- локальный контекст самого модуля формы;
- реквизиты формы, которой «принадлежит» модуль;

- свойства и методы объекта `УправляемаяФорма` встроенного языка;
- свойства и методы расширения формы, определяемого типом того объекта, данные которого содержатся в основном реквизите формы;
- глобальный контекст, в том числе неглобальные общие модули и экспортируемые функции и процедуры глобальных общих модулей;
- экспортируемые переменные, процедуры и функции модуля управляемого приложения.

Далее по порядку проверить:

1. Объявлена ли в модуле формы переменная `СтрокаТабличнойЧасти`? Нет.
2. Есть ли у формы реквизит `СтрокаТабличнойЧасти`? Нет.
3. Есть ли у объекта `УправляемаяФорма` свойство `СтрокаТабличнойЧасти`? Нет.
4. Есть ли у расширения формы свойство `СтрокаТабличнойЧасти`? Нет.
5. Есть ли свойство глобального контекста `СтрокаТабличнойЧасти`? Нет.
6. Есть ли в модуле управляемого приложения экспортная переменная `СтрокаТабличнойЧасти`? Нет.

Значит `СтрокаТабличнойЧасти` – это локальная переменная, определяемая непосредственно в этом операторе присваивания.

Как понять работу кода на встроенном языке

На предыдущем занятии мы писали код обработчика события `МатериалыКоличествоПриИзменении` (листинг 4.1) и кратко объясняли смысл написанного.

Теперь мы покажем два способа, как самому разобраться с множеством незнакомых свойств и методов объектов конфигурации, чтобы в будущем самостоятельно изучать фрагменты кода или создавать свои собственные процедуры на встроенном языке.

Синтакс-помощник – инструмент, созданный для помощи разработчику, содержащий описание всех программных объектов, которые использует система, их методов, свойств, событий и пр.

Чтобы открыть синтакс-помощник, нужно нажать соответствующую кнопку  на панели инструментов конфигулятора или выполнить команду главного меню Справка > Синтакс-помощник (рис. 5.20).

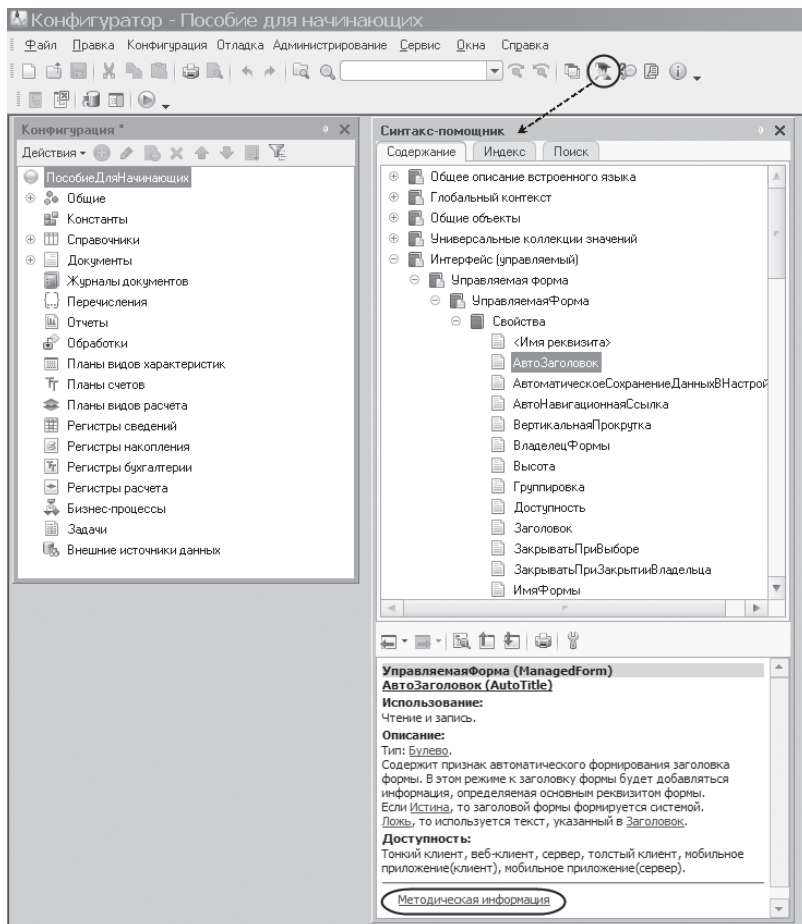


Рис. 5.20. Вызов синтакс-помощника

Как и любая другая справочная система, он представляет собой древовидную структуру, состоящую из глав, разделов, подразделов и т.п. Содержание синтакс-помощника полностью дублирует описание встроенного языка в пяти томах, входящих в стандартный комплект поставки «1С:Предприятия». Однако пользоваться синтакс-помощником, на наш взгляд, удобнее, так как он находится сразу под рукой и имеет возможность контекстной помощи (Ctrl + F1).

Кроме того, в синтакс-помощнике в конце каждого описания находится ссылка Методическая информация (см. рис. 5.20). По этой ссылке открывается окно браузера, в котором подобраны методические материалы для выбранного раздела. Источниками материалов являются: ИТС, партнерская конференция, база знаний по технологическим вопросам крупных внедрений, сайт «1С:Предприятия», конференция начинающих разработчиков и др. Ссылки на методические материалы постоянно обновляются. Таким образом, разработчики могут быстро, не отрываясь от работы, найти информацию по нужному вопросу на различных ресурсах фирмы «1С».

Анализ кода с помощью синтакс-помощника

Пользоваться синтакс-помощником удобно в тех случаях, когда нужно разобраться в уже написанном незнакомом коде. На примере нашего обработчика события МатериалыКоличествоПриИзменении (см. листинг 4.1) продемонстрируем, как понять код обработчика, используя синтакс-помощник.

Первый способ

Первый способ – найти нужный раздел в содержании и спускаться вниз по дереву, раскрывая нужные подразделы, свойства, ссылки и т.п.

Итак, перед нами первая строка нашего обработчика (листинг 5.19).

Листинг 5.19. Процедура «МатериалыКоличествоПриИзменении» (первая строка)

```
СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные;
```

Чтобы понять этот код, нужно прежде всего понимать, в каком контексте он выполняется. Программный контекст зависит от того,

в каком модуле располагается код. В данном случае процедура обработчика находится в модуле формы, следовательно, мы находимся в контексте модуля формы.

Будем изучать нашу строку последовательно слева направо. Что такое `СтрокаТабличнойЧасти`? Слева от оператора присваивания (`=`) находится либо какое-то свойство, доступное нам непосредственно в этом контексте, либо переменная.

Согласно алгоритму, изложенному в предыдущем разделе, мы должны проверить:

- Объявлена ли в модуле формы переменная `СтрокаТабличнойЧасти`? Откроем модуль формы (о том, как это сделать, рассказано на стр. 158). Мы не видим здесь строки описания переменной (`Перем СтрокаТабличнойЧасти;`), значит, это не переменная модуля формы.
- Есть ли у формы реквизит `СтрокаТабличнойЧасти`? Откроем форму документа `ПриходнаяНакладная` и перейдем в окно реквизитов формы, расположенное справа вверху редактора форм (рис. 5.21).

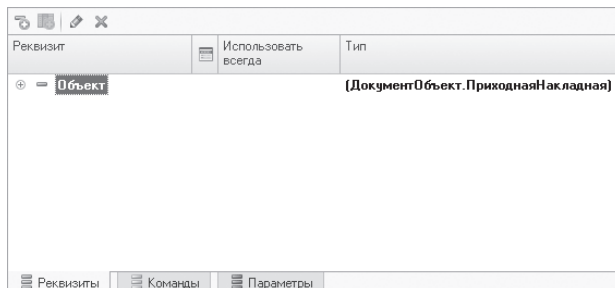


Рис. 5.21. Список реквизитов формы документа «Приходная накладная»

Мы видим, что у формы один основной (он выделен жирным шрифтом) реквизит `Объект`. Значит, реквизита `СтрокаТабличнойЧасти` у формы нет.

- Есть ли у объекта `УправляемаяФорма` свойство `СтрокаТабличнойЧасти`? Посмотрим в синтакс-помощнике свойства управляемой формы. Откроем синтакс-помощник на закладке `Содержание`. Управляемая форма – это объект интерфейса управляемого приложения, поэтому раскроем раздел `Интерфейс (управляемый)` >

Управляемая форма. Затем раскроем объект УправляемаяФорма и его Свойства (рис. 5.22).

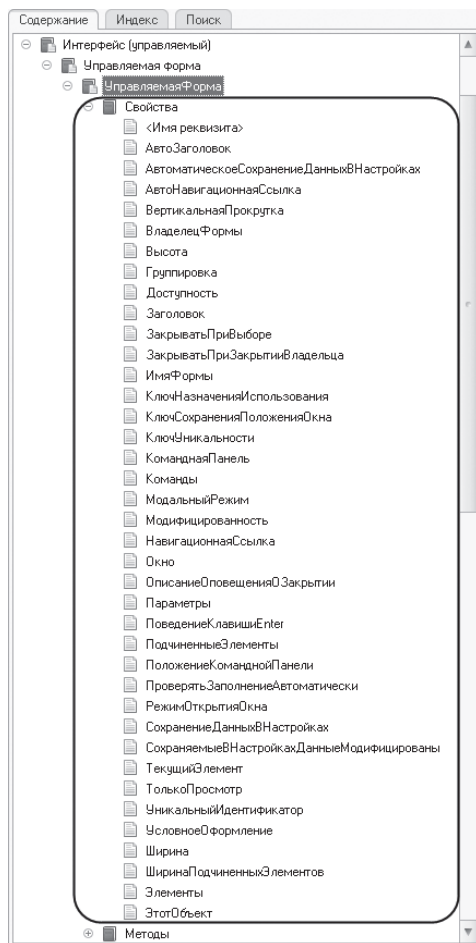


Рис. 5.22. Список свойств объекта «УправляемаяФорма» в синтаксис-помощнике

Свойства расположены в алфавитном порядке. Мы видим, что среди них нет свойства СтрокаТабличнойЧасти.

- Есть ли у расширения формы свойство СтрокаТабличнойЧасти? Мы знаем, что основной реквизит формы содержит данные объекта ДокументОбъект.ПриходнаяНакладная (см. рис. 5.21). Следовательно, в модуле формы становятся доступны свойства, методы объекта встроенного языка Расширение управляемой формы для документа (синтакс-помощник – Интерфейс (управляемый) > Управляемая форма > Расширение документа). Посмотрим на них (рис. 5.23).

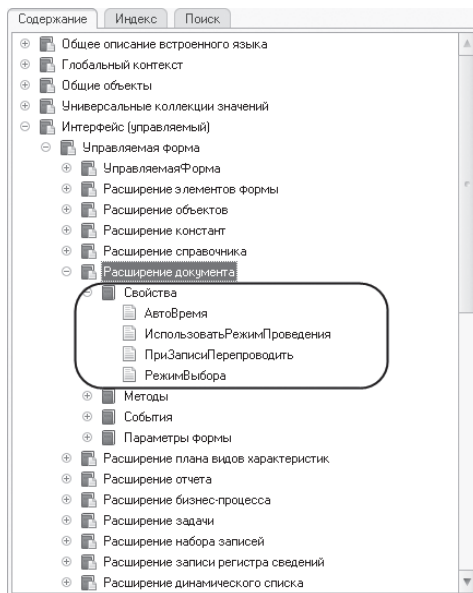


Рис. 5.23. Список свойств объекта «Расширение управляемой формы для документа» в синтакс-помощнике

Мы видим, что среди них нет свойства СтрокаТабличнойЧасти.

- Есть ли свойство глобального контекста СтрокаТабличнойЧасти? Откроем в синтакс-помощнике свойства глобального контекста (рис. 5.24).

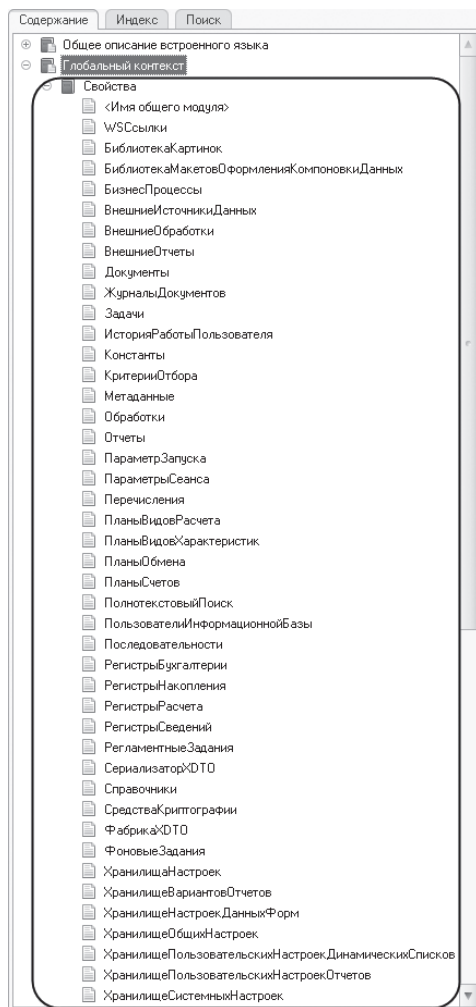


Рис. 5.24. Список свойств глобального контекста в синтакс-помощнике

Мы видим, что среди них нет свойства `СтрокаТабличнойЧасти`. Выражение `СтрокаТабличнойЧасти` также не может быть именем неглобального общего модуля, так как к его процедурам следует обращаться через точку (`СтрокаТабличнойЧасти.`). Также это выражение не может быть экспортируемой процедурой глобаль-

ного общего модуля, так как в этом случае мы могли бы только вызвать эту процедуру как `СтрокаТабличнойЧасти()`, а не присваивать ей что-то.

- Есть ли в модуле управляемого приложения экспортная переменная `СтрокаТабличнойЧасти`? Откроем модуль управляемого приложения (о том, как это сделать, рассказано на стр. 158). Мы не видим здесь строки описания переменных (`Перем СтрокаТабличнойЧасти Экспорт;`), значит, это не переменная модуля управляемого приложения.

Таким образом, понятно, что выражение `СтрокаТабличнойЧасти` – это локальная переменная процедуры `МатериалыКоличествоПриИзменении`. В процессе выполнения программы ей присваивается какое-то значение. Переменные во встроенном языке не типизированные, поэтому в любой момент ей можно присвоить значение любого типа. Если переменная локальная, то есть используется только в контексте данной процедуры, то не требуется и ее явного объявления. Она объявляется в момент первого ее использования.

Справа от оператора присваивания находится выражение `Элементы.Материалы.ТекущиеДанные`. Чтобы понять, что такое `Элементы`, пройдемся еще раз по тому же алгоритму, что и в случае с локальной переменной `СтрокаТабличнойЧасти`.

- Объявлена ли в модуле формы переменная `Элементы`? Нет.
- Есть ли у формы реквизит `Элементы`? Нет.
- Есть ли у объекта `УправляемаяФорма` свойство `Элементы`? Посмотрим еще раз на список свойств объекта `УправляемаяФорма`. Найдем в нем строку `Элементы`, значит это одно из свойств управляемой формы. Чтобы узнать, что это такое, дважды щелкнем мышью на этой строке (рис. 5.25).

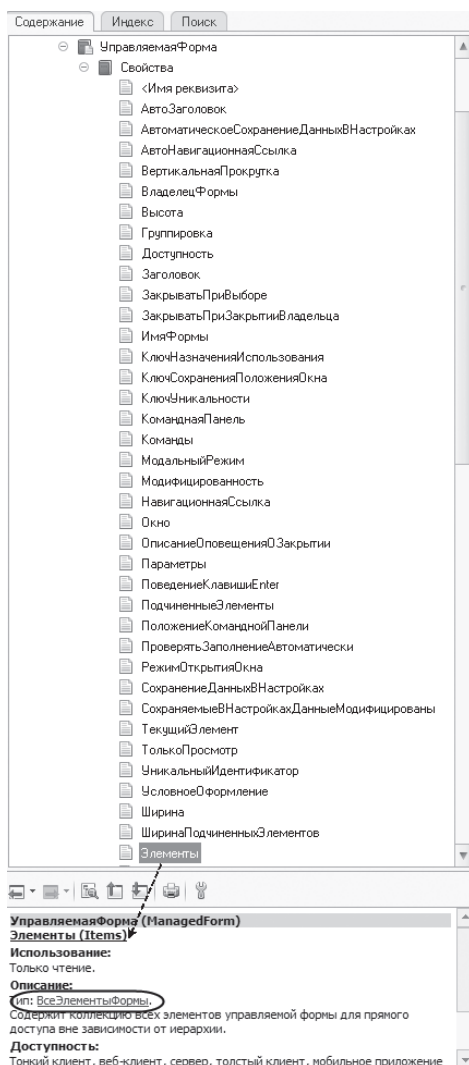


Рис. 5.25. Описание свойства «Элементы» объекта «УправляемаяФорма» в синтаксис-помощнике

В нижней части окна синтаксис-помощника появится описание выделенного нами свойства. Из этого описания следует, что, используя свойство `Элементы`, мы получаем объект `ВсеЭлементыФормы`,

который содержит коллекцию всех элементов формы. Чтобы узнать, что это такое, нажмем на соответствующую ссылку. Откроется описание коллекции **ВсеЭлементыФормы** (рис. 5.26).

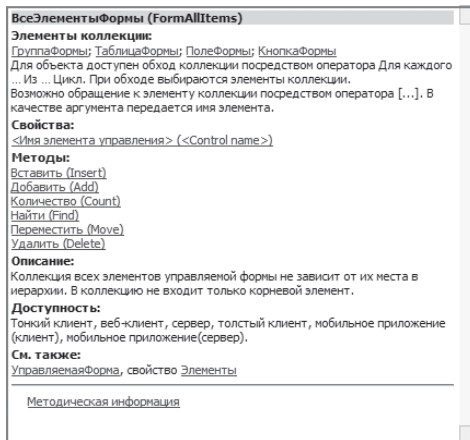


Рис. 5.26. Описание коллекции «ВсеЭлементыФормы» в синтакс-помощнике

ПРИМЕЧАНИЕ

При этом дерево синтакс-помощника, отображаемое сверху, не изменилось. Если вы хотите найти, в какой ветке дерева находится открытое сейчас описание, нужно нажать кнопку **Найти** текущий элемент в дереве , находящуюся над окном описания объектов синтакс-помощника.

Эта коллекция содержит элементы управляемой формы, размещенные на форме. Доступ к элементу осуществляется по имени.

Итак, что такое **Элементы**, мы знаем. Дальше через точку от этого объекта у нас написано: **Элементы.Материалы**. У коллекции есть такое свойство – **<имя элемента управления>**. **Материалы** – это имя некоторого элемента формы. Посмотрим на структуру элементов формы. Откроем форму документа **ПриходнаяНакладная** и перейдем в окно элементов формы, расположенное слева сверху редактора форм (рис. 5.27).

В структуре элементов формы мы видим таблицу – **Материалы**. Открыв палитру свойств этой таблицы, в заголовке мы видим – **Свойства: Таблица** (рис. 5.28).

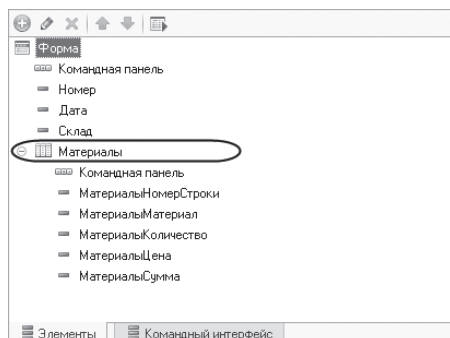


Рис. 5.27. Структура элементов формы документа «Приходная накладная»

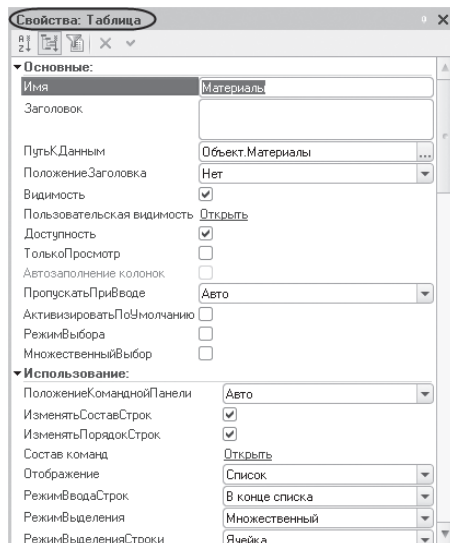


Рис. 5.28. Палитра свойств элемента формы – таблица

Значит, этот элемент формы является таблицей, и, следовательно, нам нужен объект коллекции элементов формы ТаблицаФормы. Чтобы узнать, что это такое, нажмем на соответствующую ссылку (см. рис. 5.26). Мы увидим список свойств объекта ТаблицаФормы. Свойства расположены в алфавитном порядке (рис. 5.29).

ТаблицаФормы (FormTable)
Свойства:
АвтоВводНезаполненного (AutoAddInComplete)
АвтоВводНовойСтроки (AutoInsertNewRow)
АвтоОтметкаНезаполненного (AutoMarkInComplete)
АктивизироватьПоУмолчанию (DefaultItem)
ВертикальнаяПолосаПрокрутки (VerticalScrollBar)
ВертикальныеЛинии (VerticalLines)
Видимость (Visible)
Вывод (Output)
ВыделенныеСтроки (SelectedRows)
Высота (Height)
ВысотаСтрокТаблицы (HeightInTableRows)
ВысотаЗаголовка (TitleHeight)
ВысотаПодвала (FooterHeight)
ВысотаШапки (HeaderHeight)
ГоризонтальнаяПолосаПрокрутки (HorizontalScrollBar)
ГоризонтальныеЛинии (HorizontalLines)
Доступность (Enabled)
Заголовок (Title)
ИзменитьПорядокСтрок (ChangeRowOrder)
ИзменитьСоставСтрок (ChangeRowSet)
Имя (Name)
КартинкаСтрок (RowsPicture)
КоманднаяПанель (CommandBar)
КонтекстноеМеню (ContextMenu)
МножественныйВыбор (MultipleChoice)
НачальноеОтображениеДерева (InitialTreeView)
НачальноеОтображениеСписка (InitialListView)
ОтметкаНезаполненного (MarkInComplete)
Отображение (Representation)
ОтображениеПанелиПоиска (SearchPanelRepresentation)
ОтображениеПодсказки (ToolTipRepresentation)
Подвал (Footer)
Подсказка (ToolTip)
ПодчиненныеЭлементы (ChildItems)
ПоискПоВводу (SearchOnInput)
ПоложениеЗаголовка (TitleLocation)
ПоложениеКоманднойПанели (CommandBarLocation)
ПропускатыПоВводу (SkipOnInput)
ПутьКДанным (DataPath)
ПутьКДаннымКартинкиСтроки (RowPictureDataPath)
РазрешитьНачалоПеретаскивания (EnableStartDrag)
РазрешитьПеретаскивание (EnableDrag)
РастягиватьПоВертикали (VerticalStretch)
РастягиватьПоГоризонтали (HorizontalStretch)
РасширеннаяПодсказка (ExtendedToolTip)
РежимВводаСтрок (RowInputMode)
РежимВыбора (ChoiceMode)
РежимВыделения (SelectionMode)
РежимВыделенияСтроки (RowSelectionMode)
Родитель (Parent)
СочетаниеКлавиш (Shortcut)
ТекущаяСтрока (CurrentRow)
ТекущиеДанные (CurrentData)
ТекущийРодитель (CurrentParent)
ТекущийЭлемент (CurrentItem)
ТолькоПросмотр (ReadOnly)
ЦветРамки (BorderColor)
ЦветТекста (TextColor)
ЦветТекстаЗаголовка (TitleTextColor)
ЦветФона (BackColor)
ЧередованиеЦветовСтрок (UseAlternationRowColor)
Шапка (Header)
Ширина (Width)
Шрифт (Font)
ШрифтЗаголовка (TitleFont)

Рис. 5.29. Список свойств объекта «ТаблицаФормы» в синтакс-помощнике

Итак, что такое **Элементы.Материалы**, мы знаем. Дальше через точку от этого объекта у нас написано: **Элементы.Материалы.ТекущиеДанные**. Прокрутив список свойств таблицы управляемой формы вниз, мы видим свойство **ТекущиеДанные**. Значит, это одно из свойств объекта **ТаблицаФормы**. Чтобы узнать, что это такое, дважды щелчком мышью на этой строке (рис. 5.30).

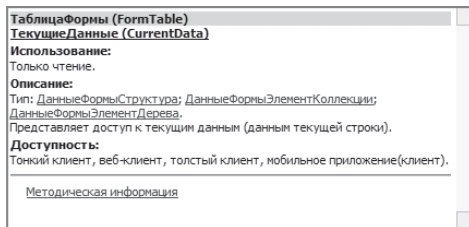


Рис. 5.30. Описание свойства «Текущие Данные» объекта «ТаблицаФормы» в синтакс-помощнике

В нижней части окна синтакс-помощника появится описание выделенного нами свойства. Из этого описания следует, что, используя свойство `ТекущиеДанные`, мы получаем объект `ДанныеФормыСтруктура`, который содержит данные, находящиеся в текущей строке таблицы.

Значит, в результате выполнения первой строки обработчика `СтрокаТабличнойЧасти = Элементы.Материалы.ТекущиеДанные` в переменной `СтрокаТабличнойЧасти` у нас окажется объект типа `ДанныеФормыСтруктура`.

Теперь посмотрим на следующую строку обработчика (листинг 5.20).

Листинг 5.20. Процедура «МатериалыКоличествоПриИзменении» (вторая строка)

```
СтрокаТабличнойЧасти.Сумма = СтрокаТабличнойЧасти.Количество *
                               СтрокаТабличнойЧасти.Цена;
```

Логично предположить, что `Сумма`, `Количество` и `Цена` – это какие-то свойства объекта `ДанныеФормыСтруктура`, который после выполнения первой строки находится в переменной `СтрокаТабличнойЧасти`. Чтобы узнать, что это такое, нажмем на соответствующую ссылку (см. рис. 5.30, 5.31).

Из описания этого объекта следует, что, используя объект `ДанныеФормыСтруктура`, мы можем обратиться к данным конкретной колонки табличной части, указав имя колонки в качестве свойства объекта. То есть, используя выражение `СтрокаТабличнойЧасти.Сумма`, мы обращаемся к данным, которые находятся в колонке `Сумма` текущей строки таблицы. И в них помещается произведение данных, содержащихся в колонке `Количество`, на данные в колонке `Цена`.

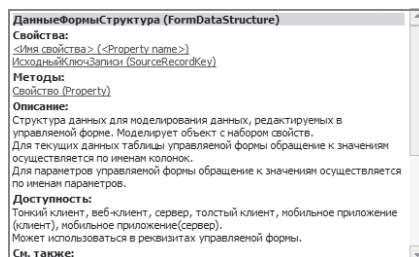


Рис. 5.31. Описание объекта «ДанныеФормыСтруктура» в синтакс-помощнике

Второй способ

Второй способ – воспользоваться контекстной помощью синтакс-помощника. Для этого нужно открыть программный модуль, установить курсор на интересующую вас конструкцию встроенного языка и нажать одновременно клавиши Ctrl и F1. Откроем форму документа ПриходнаяНакладная, перейдем на закладку Модуль, откроем текст процедуры МатериалыКоличествоПриИзменении.

Немного выше мы установили, что выражение СтрокаТабличнойЧасти слева от оператора присваивания – это локальная переменная.

Справа от оператора присваивания находится выражение Элементы.Материалы.ТекущиеДанные. Установим курсор на выражении Элементы и нажмем Ctrl + F1. Среди конструкций встроенного языка, отсортированных по алфавиту, будет произведен поиск выражения Элементы (рис. 5.32).

Мы видим, что выражение найдено, так как появилось окно со списком глав, в которых это выражение используется. Каждая глава в списке содержит полный путь к искомому выражению, начиная от корня структуры содержания синтакс-помощника. Уровни этой структуры отделены друг от друга символом «/» (слеш). Таким образом, подведя мышь к какому-либо элементу списка глав, мы сможем оценить, та ли это глава, которая нам нужна.

Однако список глав, в которых используется искомое выражение, достаточно большой, и вручную искать нужную главу довольно утомительно. Чтобы найти свойство Элементы, которое используется в управляемой форме, нажмем Ctrl + F, находясь в окне списка, и введем в окно поиска выражение УправляемаяФорма. Нажмем кнопку Искать. После этого в списке глав будет выделена нужная глава (рис. 5.33).

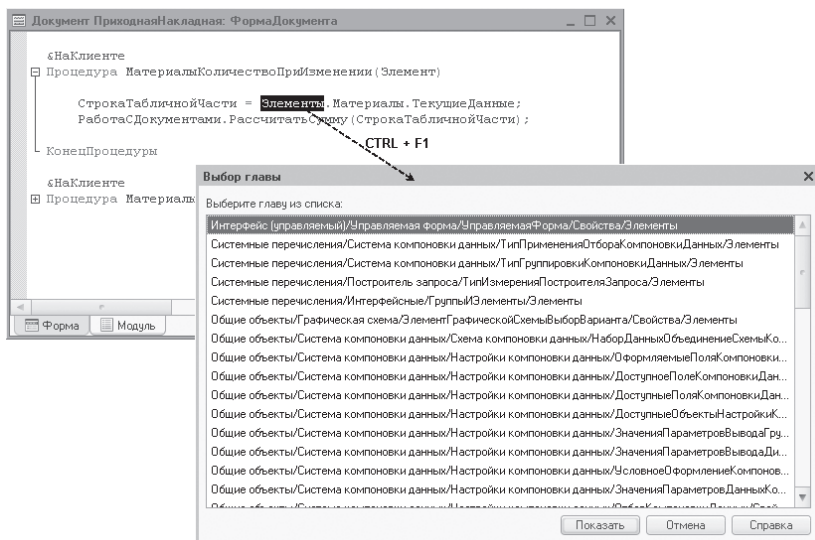


Рис. 5.32. Вызов контекстной помощи в синтакс-помощнике

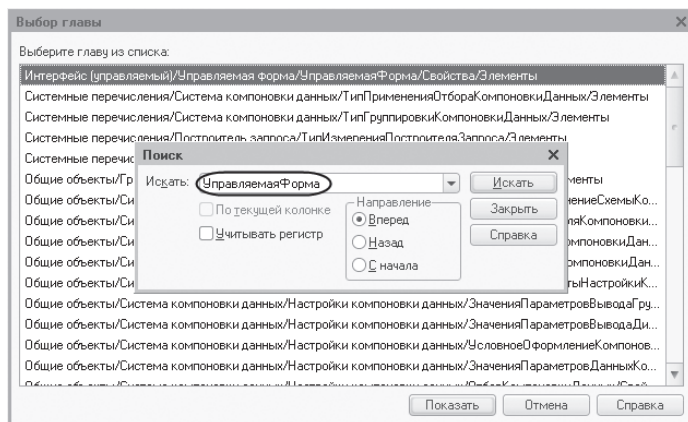


Рис. 5.33. Поиск в списке глав при контекстном поиске

Нажмем кнопку Показать. Описание выбранной главы откроется в нижнем окне синтакс-помощника (рис. 5.34). При этом дерево синтакс-помощника, отображаемое вверху, не изменится. Чтобы найти, в какой ветке дерева находится открытое сейчас описание,

нажмем кнопку Найти текущий элемент в дереве , находящуюся над окном описания объектов синтаксис-помощника.

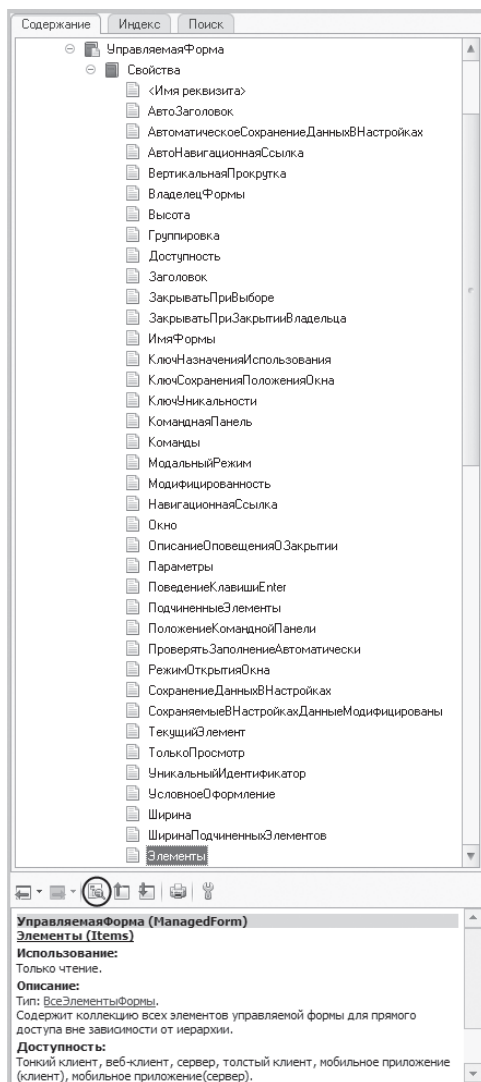


Рис. 5.34. Описание свойства «Элементы» объекта «УправляемаяФорма»

На закладке **Содержание** синтакс-помощника будет показан раздел, соответствующий текущему описанию. Таким образом, в дереве содержания мы видим, что **Элементы** – это свойство объекта **УправляемаяФорма**. Далее, раскрывая соответствующие ссылки, как описано в первом способе работы с синтакс-помощником, мы поймем, что содержится в переменной **СтрокаТабличнойЧасти** и как можно обращаться к данным в ее колонках. И так далее.

Есть также еще одна полезная возможность использования синтакс-помощника. Можно ограничить состав объектов, которые будут отображаться в нем. Так как мы находимся на клиенте, в форме, имеет смысл ограничиться только объектами встроенного языка, доступными в режимах **Тонкий клиент** и **Веб-клиент**. Для этого нужно выполнить команду главного меню **Сервис > Параметры** или нажать кнопку **Открыть режим настройки параметров** , находящуюся над окном описания объектов синтакс-помощника. На закладке **Справка** окна **Параметры** можно снять или поставить отметку у нужных режимов исполнения (рис. 5.35).

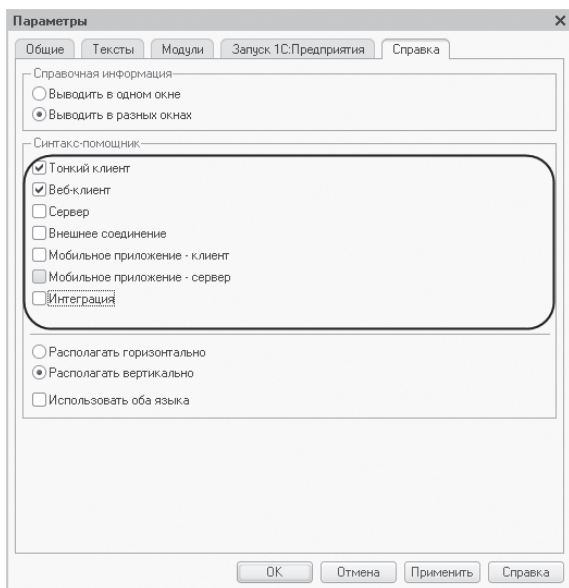


Рис. 5.35. Фильтрация объектов, показываемых в синтакс-помощнике

ВНИМАНИЕ!

Когда вы пишете процедуру, исполняемую, например, на сервере, и хотите воспользоваться синтакс-помощником, не забудьте установить соответствующий флажок режима исполнения для отображения объектов встроенного языка.

Анализ кода с помощью отладчика

Пользоваться отладчиком наиболее удобно в тех случаях, когда нужно написать какой-то собственный код. Потому что в отличие от синтакс-помощника, где нужно, вообще говоря, хорошо представлять контексты исполнения, структуру объектов и пр., с помощью отладчика ничего этого представлять не нужно.

Можно просто остановиться в конкретном месте программы и посмотреть, какие же свойства здесь доступны или какие программные объекты здесь используются.

Отладчик – вспомогательный инструмент, облегчающий разработку и отладку программных модулей системы «1С:Предприятие». Отладчик предоставляет следующие возможности:

- пошаговое выполнение модуля,
- расстановка точек останова,
- прерывание и продолжение выполнения модуля,
- возможность отладки нескольких модулей одновременно,
- вычисление выражений для анализа состояния переменных,
- просмотр стека вызовов процедур и функций,
- возможность остановки по возникновению ошибки,
- возможность редактирования модуля в процессе отладки.

Но мы пока не будем подробно останавливаться на всех этих возможностях, а рассмотрим использование отладчика для того, чтобы разобраться с обработчиком события **МатериалыКоличествоПриИзменении**, приведенном в листинге 4.1.

Если в режиме Конфигуратор редактируется текст модуля, то становятся доступными команды пункта главного меню **Отладка**, позволяющие расставлять и убирать точки останова. *Точки останова* позволяют прерывать выполнение программы в тех местах, где они установлены. Затем разработчик может проанализировать значение и тип выражений и переменных модуля в момент остановки и продолжить выполнение программы до следующей точки останова и т. д.

Итак, откроем форму документа ПриходнаяНакладная, перейдем на закладку Модуль, откроем текст процедуры МатериалыКоличествоПриИзменении. Мы видим, что в пункте главного меню Отладка и на панели инструментов конфигуратора стали доступны команды для работы с точками останова (рис. 5.36).



Рис. 5.36. Панель инструментов «Точки останова»

Установить точку останова можно двойным щелчком мыши в служебной области слева от нужной строки. При этом в служебной области, располагающейся вдоль левой границы, появится значок точки останова.

ПРИМЕЧАНИЕ

Для установки точки останова можно также установить указатель в любое место строки модуля, в которой требуется выполнить остановку, и выполнить команду Отладка > Точка останова или нажать соответствующую кнопку  на панели инструментов Точки останова.

Снять точку останова можно двойным щелчком мыши на этой точке в служебной области.

ПРИМЕЧАНИЕ

Для снятия точки останова можно также установить указатель в любое место строки модуля и выполнить команду Отладка > Отключить точку останова или нажать соответствующую кнопку  на панели инструментов Точки останова.

Для снятия всех точек останова нужно выполнить команду Отладка > Отключить все точки останова или нажать соответствующую кнопку на панели инструментов Точки останова.

Итак, дважды щелкнем в служебной области слева от первой строки процедуры МатериалыКоличествоПриИзменении (рис. 5.37).

Чтобы иметь возможность отлаживать код на встроенном языке, нужно обеспечить запуск приложения, в котором выполняется код, в отладочном режиме. Для начала отладки выполним команду Отладка > Начать отладку или нажмем соответствующую кнопку на панели инструментов конфигуратора. Конфигуратор запустит «1С:Предприятие» в отладочном режиме. На самом деле мы так делали и раньше, но не устанавливали точек останова, и программа не прерывалась.

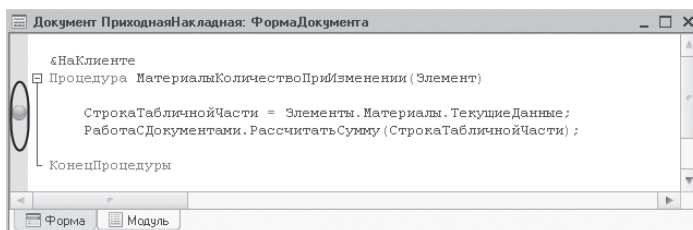


Рис. 5.37. Установка точки останова в процедуре «МатериалыКоличествоПриИзменении»

Откроем список документов Приходные накладные и откроем любой из двух созданных нами документов. Изменим поле Количество в любой строке документа. Выполнение программы прервется, и в конфигураторе будет открыта процедура МатериалыКоличествоПриИзменении в точке останова. Рядом с ней появится стрелка, указывающая на текущую исполняемую строку модуля (рис. 5.38).

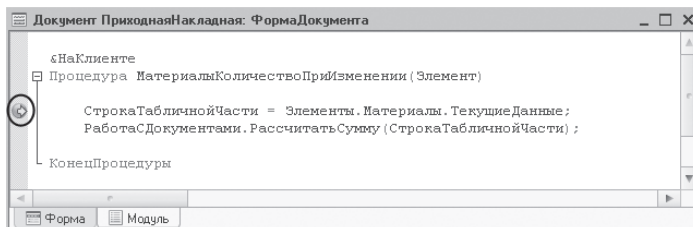


Рис. 5.38. Остановка программы в точке останова

Также мы видим, что в пункте главного меню Отладка и на панели инструментов конфигуратора стали доступны команды для работы с конфигурацией в процессе отладки (рис. 5.39).



Рис. 5.39. Панель инструментов «Отладка конфигурации»

С помощью этих команд и кнопок на панели инструментов Отладка конфигурации можно пошагово выполнять программу (кнопки Шагнуть через, Шагнуть в, Шагнуть из) или продолжить отладку (кнопка Продолжить отладку) до следующей точки останова. С помощью кнопок Табло и Вычислить выражение можно получить значения интересующих вас выражений в каждый момент остановки

программы. С помощью кнопки **Стек вызовов** можно проследить последовательность вызова процедур и функций.

Но мы сейчас уже остановились в интересующей нас строке процедуры **МатериалыКоличествоПриИзменении**. Заметим, что эта строка еще не выполнялась, поэтому значения переменных будут еще не заполнены. Чтобы увидеть их после выполнения строки, можно нажать кнопку **Шагнуть через**.

Итак, мы хотим понять, что означает выражение **Элементы.Материалы.ТекущиеДанные** и что содержит этот объект в момент остановки программы. Двойным щелчком выделим слово **Элементы** и нажмем кнопку **Вычислить выражение (Shift + F9)**  на панели инструментов **Отладка конфигурации**.

В поле **Выражение** попадет выделенное нами слово **Элементы**. В соответствующих колонках мы увидим значение и тип этого объекта. Мы видим, что объект **Элементы** является коллекцией значений **ВсеЭлементыФормы**, содержащей все элементы формы. Раскроем этот объект (рис. 5.40).

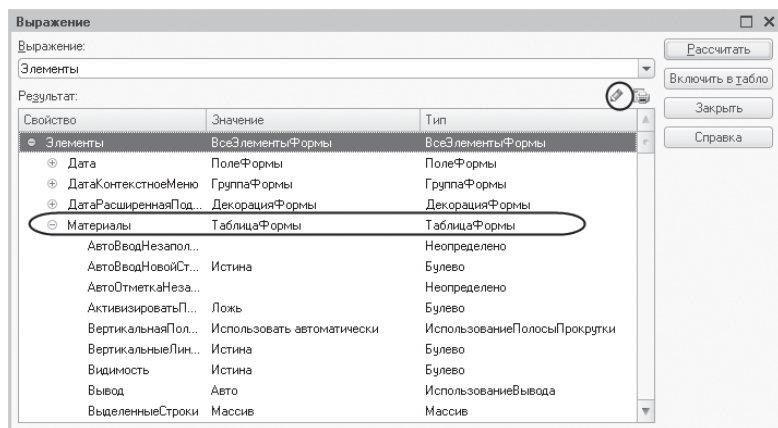


Рис. 5.40. Объект «Элементы»

Над окном **Результат** находится кнопка **Показать значения в отдельном окне**, по нажатию которой (или F2) можно будет просматривать содержимое коллекций.

Дальше нас интересует выражение **Материалы**. Найдем его в списке элементов формы. Мы видим, что это объект **ТаблицаФормы**.

Раскроем этот объект и увидим его свойства. Нас интересует свойство `ТекущиеДанные`. Найдем в списке и раскроем этот объект. Мы увидим данные текущей строки нашей табличной части, их значение и тип. Причем `Количество` содержит только что измененное нами значение (рис. 5.41).

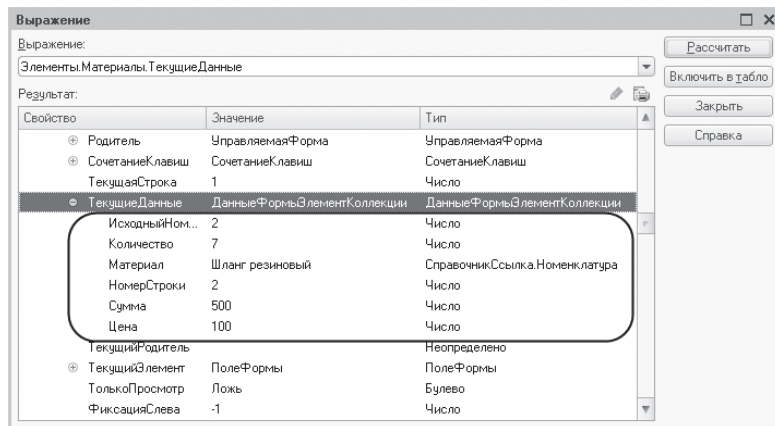


Рис. 5.41. Объект «Элементы.Материалы.ТекущиеДанные»

Закроем окно `Выражение`.

Теперь нам нужно шагнуть в процедуру общего модуля `РаботаСДокументами`, где вычисляется значение переменной `СтрокаТабличнойЧасти`. Для этого нажмем кнопку `Шагнуть` в на панели инструментов Отладка конфигурации. Программа перейдет в процедуру `РассчитатьСумму` общего модуля `РаботаСДокументами`. Двойным щелчком выделим выражение `СтрокаТабличнойЧасти` и нажмем кнопку `Вычислить` выражение. Раскроем объект `СтрокаТабличнойЧасти` (рис. 5.42).

Мы видим, что теперь переменная `СтрокаТабличнойЧасти` содержит объект `ДанныеФормыЭлементКоллекции`. Но значения колонки `Сумма` еще не пересчитаны, так как вторая строка кода еще не исполнялась. Закроем окно `Выражение` и еще раз нажмем кнопку `Шагнуть` через на панели инструментов Отладка конфигурации. Программа выполнит процедуру `РассчитатьСумму` общего модуля `РаботаСДокументами` и остановится в ее конце. Теперь можно подвести курсор к колонке `Количество` или `Сумма`, и система во всплывающей подсказке покажет текущее значение (рис. 5.43).

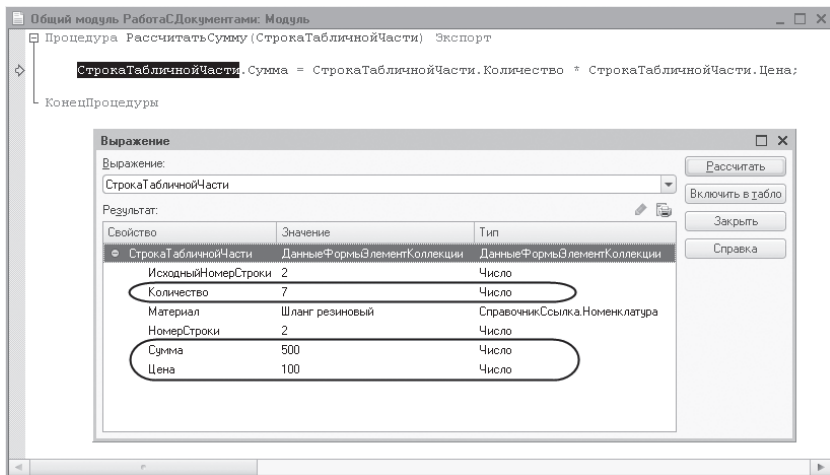


Рис. 5.42. Объект «СтрокаТабличнойЧасти»

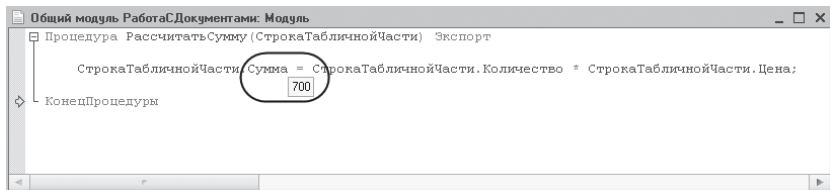


Рис. 5.43. Объект «СтрокаТабличнойЧасти»

Мы видим, что значения колонки Сумма пересчитались по заданному алгоритму, и, значит, объект `Элементы.Материалы.ТекущиеДанные` содержит новые значения.

Таким образом, можно поставить точку останова в тело цикла и смотреть значения переменных по мере его выполнения и т. д.

После анализа переменных и выражений в момент остановки можно продолжить отладку, нажав кнопку `Продолжить отладку`. Если необходимо прервать отладку, нужно выполнить команду `Отладка > Завершить`. В процессе отладки допускается редактирование текущей конфигурации и сохранение изменений.

ВНИМАНИЕ!

Хотя в процессе отладки возможно редактирование отлаживаемого модуля, отладчик не производит компилирование измененного кода – продолжается отладка кода конфигурации базы данных (на момент запуска отладчика или подключения). Для отладки изменений, внесенных в конфигурацию, необходимо выполнить обновление конфигурации базы данных.

В заключение скажем о следующих полезных приемах работы с отладчиком.

Когда вы находитесь в модуле формы и вам нужно написать какой-то обработчик, можно использовать свойство ЭтотОбъект, чтобы посмотреть свойства контекста этой конкретной формы со всеми теми расширениями, которые у нее есть и пр. Для этого, после того как вы остановились в какой-либо точке останова в модуле формы, нужно нажать кнопку Вычислить выражение (Shift + F9)  на панели инструментов Отладка конфигурации. В поле Выражение ввести слово ЭтотОбъект и нажать кнопку Рассчитать (рис. 5.44).

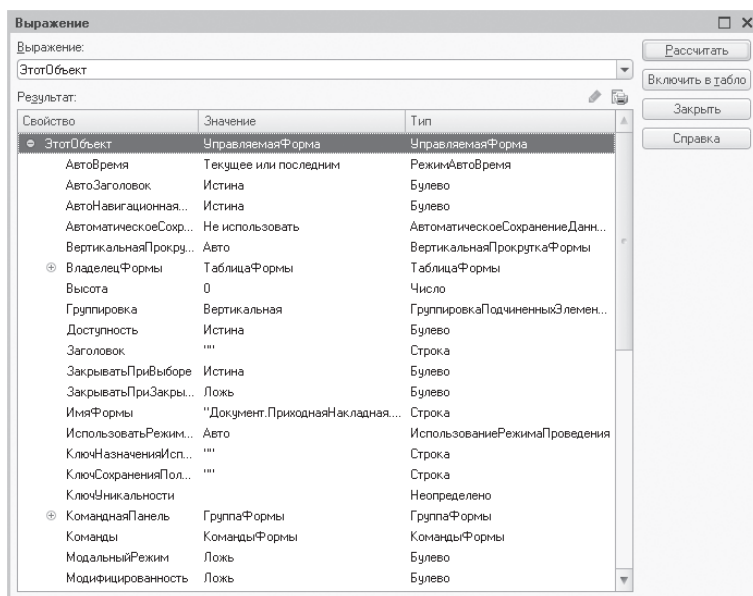


Рис. 5.44. Свойство управляемой формы «ЭтотОбъект» в отладчике

Раскрыв этот объект, вы увидите тип и свойства объектов встроенного языка, которые используются в момент останова.

Аналогичным образом, когда вы находитесь в модуле объекта или набора записей, можно использовать свойство `ЭтотОбъект`, чтобы посмотреть свойства контекста модуля объекта или набора записей.

Объекты, объекты, объекты...

Что такое объект в терминах «1С:Предприятия»? Этот вопрос зачастую ставит в тупик не только начинающих разработчиков, но и людей, имеющих определенный опыт разработки на платформе «1С:Предприятие».

Основная трудность заключается в том, что всегда нужно ясно представлять себе, в каком контексте употребляется этот термин.

Как правило, термин *объект* употребляется в одном из трех контекстов:

- конфигурация,
- база данных,
- встроенный язык.

Говоря о конфигурации, термином *объект конфигурации* мы обозначаем некоторую совокупность описания данных и алгоритмов работы с этими данными. Например, в конфигурации может существовать объект *Справочник Сотрудники*.

На основании каждого объекта конфигурации в базе данных создается информационная структура, в которой будут храниться данные.

Так вот, когда мы говорим о базе данных, термином «объект» мы обозначаем всего лишь некий элемент такой информационной структуры. Характерной особенностью такого элемента является то, что на него (как на совокупность данных) существует ссылка, которая может являться значением какого-либо поля другой информационной структуры.

Например, в базе данных существует справочник *Сотрудники*, в котором есть сотрудник *Иванов*. В этом случае элемент справочника, содержащий информацию о сотруднике *Иванове*, будет являться объектом базы данных. И если в документе *Приходная Накладная* будет существовать реквизит *Ответственное Лицо*, то тип значения

этого реквизита будет ссылкой на объект базы данных, то есть на элемент справочника, содержащий информацию об Иванове.

Если же мы начинаем говорить о встроенном языке и о том, каким образом средствами встроенного языка работать со справочниками, то термином «объект» мы обозначаем тип данных, позволяющий получить доступ к данным и обладающий набором свойств и методов.

Существует целый ряд объектов встроенного языка, позволяющих работать со справочниками (СправочникиМенеджер, СправочникМенеджер.<имя>, СправочникСсылка.<имя> и т.д.). Среди них есть один объект, который предоставляет доступ к объекту справочника в базе данных, – СправочникОбъект.<имя> (рис. 5.45).

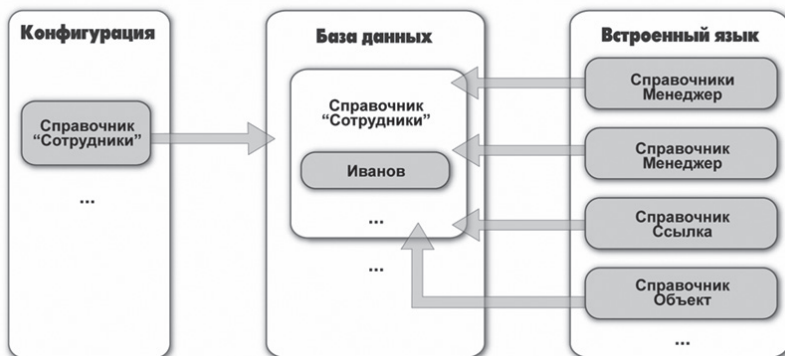


Рис. 5.45. Объект конфигурации, объект базы данных, объекты встроенного языка

Сервер и клиенты

На предыдущем занятии мы создали одну процедуру для обработки нескольких событий и поместили ее в общий модуль РаботаСДокументами. В свойствах этого общего модуля мы устанавливали флажки Клиент и Сервер. Объясним подробнее, откуда в «1С:Предприятии» взялись вообще какие-то «клиенты» и «серверы».

Система «1С:Предприятие» поддерживает два варианта работы системы: файловый и клиент-серверный.

Файловый вариант работы с информационной базой рассчитан на персональную работу одного пользователя или работу

небольшого количества пользователей в локальной сети. В этом варианте все данные информационной базы (конфигурация, база данных, административная информация) располагаются в одном файле (рис. 5.46).



Рис. 5.46. Файловый вариант работы

Основное назначение файлового варианта – быстро и легко установить систему и работать с ней. Например, чтобы что-то посмотреть или доработать дома на ноутбуке. В файловом варианте тоже можно вести реальную учетную работу, но при этом нужно понимать, что он не предоставляет абсолютно всех тех же возможностей по масштабируемости, защите данных, какие имеет клиент-серверный вариант. Поэтому, если вы самостоятельно ведете бухгалтерский учет или у вас небольшой коллектив сотрудников, и вам не требуется гарантированная защита данных от несанкционированного использования сотрудниками, и вы имеете относительно небольшой объем данных, можно работать в файловом варианте. В остальных же случаях нужно использовать клиент-серверный вариант.

Клиент-серверный вариант предназначен для использования в рабочих группах или в масштабе предприятия. Он реализован на основе трехуровневой архитектуры «клиент-сервер» (рис. 5.47).

Клиент-серверный вариант работы – это основной вариант для работы в многопользовательской среде с большим объемом данных. Он предоставляет абсолютно все возможности по масштабируемости, администрированию и защите данных. Однако он требует значительных усилий по установке и администрированию.



Рис. 5.47. Трехуровневая архитектура «1С:Предприятия»

При этом физически серверная и клиентские части системы «1С:Предприятие» могут располагаться как на разных компьютерах, так и на одном. Главное, что пользователь не имеет непосредственного доступа к серверу баз данных, и это позволяет обеспечивать безопасность данных. А в файловом варианте база данных должна находиться на некотором общем сетевом ресурсе, доступном всем пользователям.

Система «1С:Предприятие» изначально рассчитана на клиент-серверный вариант работы. Хотя сейчас вы разрабатываете свою учебную конфигурацию в файловом варианте работы, она будет работать и в клиент-серверном варианте без ваших дополнительных усилий.

Прикладные решения разрабатываются один раз и одинаково работают, что в одном, что в другом варианте. То есть переход с одного варианта на другой не требует переделки конфигурации.

Это достигается за счет того, что конфигурация разрабатывается всегда исходя из клиент-серверной архитектуры. В системе «1С:Предприятие» просто нет возможности разрабатывать ее по-другому. И в том случае, когда используется файловый вариант работы, система при исполнении прикладного решения просто «имитирует» наличие сервера на клиентском компьютере.

Клиент-серверная архитектура разделяет всю работающую систему на три различные части, определенным образом взаимодействующие между собой, – клиент, сервер «1С:Предприятия» и сервер баз данных.

Клиентское приложение – это программа, часть системы «1С:Предприятие». Основное ее назначение – организация пользовательского интерфейса, отображение данных с возможностью их изменения. Кроме этого, клиентское приложение может исполнять код на встроенном языке (то есть какие-то алгоритмы разработчика), но оперирует при этом лишь очень ограниченным пространством типов встроенного языка. Такой подход позволяет клиентскому приложению быть очень «легким», не требовать много ресурсов, «путешествовать» по Интернету и работать даже в среде веб-браузеров.

Клиентское приложение взаимодействует с сервером «1С:Предприятия». *Сервер «1С:Предприятия»* – это тоже программа, часть системы «1С:Предприятие».

Одна из основных задач этой программы – передавать запросы от клиентского приложения к серверу баз данных и возвращать обратно клиенту результаты этих запросов.

Другая задача сервера – исполнение большинства алгоритмов на встроенном языке, подготовка данных для отображения форм, отчетов и т. д. То есть все сложные вычисления, требующие непосредственной работы с данными, исполняются именно на сервере. При этом на сервере доступно практически все пространство типов встроенного языка «1С:Предприятия» (за исключением, естественно, чисто интерфейсных типов, потому что у сервера нет никакой интерфейсной части, так как он общается не с пользователями, а только с другими программами: клиентским приложением и с сервером баз данных).

Сервер баз данных – это тоже программа. Она уже не является частью системы «1С:Предприятие», это специализированная программа, поставляемая сторонними производителями. Ее основное назначение – это организация и ведение баз данных – структурированных

организованных наборов данных, описывающих характеристики каких-либо физических или виртуальных систем.

В настоящее время система «1С:Предприятие» может работать со следующими серверами баз данных:

- Microsoft SQL Server,
- PostgreSQL,
- IBM DB2,
- Oracle Database.

Компиляция общих модулей

На предыдущем занятии мы создали одну процедуру для обработки нескольких событий и поместили ее в общий модуль РаботаСДокументами. У этого модуля, как и у всякого общего модуля конфигурации, существует набор свойств: Клиент (управляемое приложение), Сервер и Внешнее соединение. Значения этих свойств (истина/ложь) определяют, где будут скомпилированы экземпляры этих модулей.

Расскажем подробнее о том, что происходит, когда мы устанавливаем те или иные флажки у общего модуля.

Прежде всего, необходимо понимать, зачем необходима компиляция. Дело в том, что все, что мы разработали и написали в конфигурации, – пока только некая «заготовка». Платформа, когда мы запускаем ее в режиме 1С:Предприятие, превращает все это в программу, которую уже можно исполнить на компьютере, – компилирует. При этом, как мы уже сказали ранее, есть разные части системы, в которых исполняется код – сервер, клиентские приложения. Поэтому для общих модулей мы можем и должны в явном виде указать, где, на какой «стороне» они должны быть скомпилированы – на сервере или клиенте.

В общем модуле РаботаСДокументами мы установили свойство Клиент (управляемое приложение), см. рис. 4.24. Это значит, что экземпляры этого модуля будут скомпилированы только на стороне клиента в контекстах тонкого клиента и веб-клиента (рис. 5.48).

Если у модуля установлено только свойство Сервер, то модуль будет скомпилирован только на стороне сервера.



Рис. 5.48. Компиляция общего модуля на стороне сервера или клиента

Клиентских приложений два: Тонкий клиент и Веб-клиент.

Тонкий клиент – это файл 1cv8c.exe. Именно его запускает платформа автоматически, когда мы начинаем отладку.

Веб-клиент не существует в виде файла, потому что он исполняется не в среде операционной системы, а в среде интернет-браузера. Пользователю достаточно всего лишь запустить свой браузер, ввести адрес веб-сервера, на котором опубликована информационная база, и веб-клиент скачается к нему на компьютер и начнет исполняться.

Директивы компиляции

На предыдущем занятии мы создали обработчики событий ПриИзменении у некоторых элементов формы. Когда система создавала объявления процедур, в которых располагается код этих обработчиков, она предваряла их директивами &НаКлиенте. Расскажем подробнее о том, что это значит.

Дело в том, что форма существует и на сервере, и на клиенте одновременно. Поэтому для каждой процедуры, которая существует в модуле формы, нужно указывать в явном виде контекст ее исполнения, где она будет исполняться: на сервере или на клиенте. Наличие директив &НаКлиенте, или &НаСервере, или &НаСервереБезКонтекста связано с тем, что при использовании встроенного языка в модуле формы клиентский код должен быть четко отделен от серверного. Таким образом, указав одну из директив, разработчик в явном виде программирует серверную или клиентскую части.

В модуле формы одновременно можно поместить процедуры с различными директивами исполнения и передавать выполнение кода с клиента на сервер. Также из клиентской процедуры модуля формы можно вызвать процедуру общего модуля (как и сделано в нашем примере), которая может выполняться как на клиенте, так и на сервере, в зависимости от установленных свойств модуля Клиент или Сервер.

Исполнение кода на клиенте и на сервере

На предыдущем занятии мы создали одну процедуру для обработки нескольких событий и поместили ее в общий модуль РаботаСДокументами. При этом в его свойствах мы поставили флажок Клиент (управляемое приложение) и сняли флажок Сервер. Эта процедура вызывалась из модуля формы, в котором для обработчика события также была указана директива, определяющая контекст исполнения этого обработчика – &НаКлиенте. Расскажем подробнее, как именно происходит исполнение кода в этом случае.

После запуска прикладного решения выполнение кода всегда начинается на клиенте. В процессе работы выполнение кода может быть передано на сервер посредством вызова процедуры общего модуля, скомпилированного на сервере.

Дело в том, что при вызове процедуры или функции ее поиск осуществляется сначала на клиенте. Если скомпилированный контекст клиента не содержит данную процедуру, то поиск продолжается на стороне сервера. Если вызываемая процедура будет найдена, то выполнение кода будет передано на сервер. После завершения процедуры выполнение кода продолжится на клиенте.

Например, при изменении цены в форме приходной накладной вызывается процедура модуля формы ЦенаПриИзменении(). Она исполняется на стороне клиента в режиме тонкого клиента. Во второй строке этого обработчика вызывается процедура РассчитатьСумму(). Эта процедура находится в общем модуле РаботаСДокументами, скомпилированном также на клиенте. Она исполняется в режиме тонкого клиента. После выполнения этой процедуры управление опять передается в модуль формы (рис. 5.49).

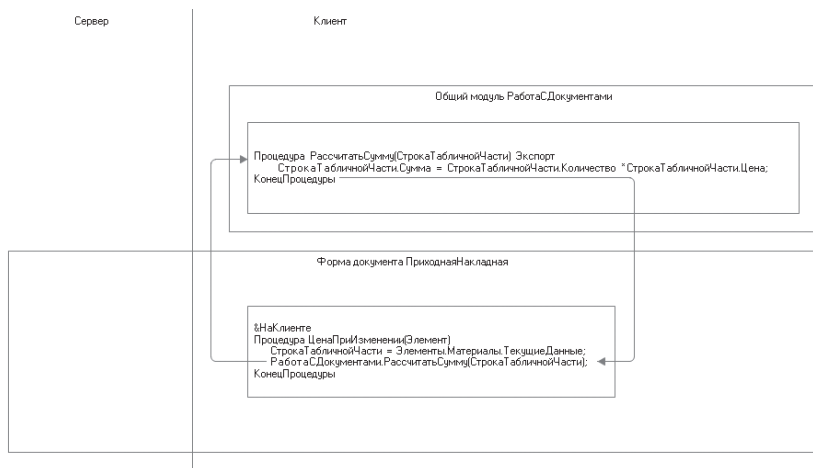


Рис. 5.49. Передача выполнения кода между модулями на клиенте